

BACKEND · SATC 2026.1

Aula 02

Ambiente de Desenvolvimento: Java na prática + Git na prática

1

Java na prática

Executando e explorando o projeto ola-mundo no IntelliJ IDEA

2

Git na prática

Instalando o Git, configurando SSH e enviando o primeiro commit ao GitHub

O que vamos fazer hoje

01

Preparar o ambiente

Conferir se o JDK e o IntelliJ IDEA estão instalados na máquina.

02

Explorar o projeto ola-mundo

Abriu, executar e mexer no primeiro projeto Java, direto no IntelliJ.

03

Instalar e configurar o Git

Instalar o Git, criar conta no GitHub e configurar acesso via SSH.

04

Versionar o projeto

Fazer o primeiro commit do ola-mundo e enviar para o GitHub.

PARTE 1

Java na prática

Vamos abrir o IntelliJ IDEA, rodar o projeto ola-mundo pela primeira vez e entender cada linha do código.



Antes de começar: pré-requisitos

1

JDK instalado

O projeto usa Java na versão 25. Confirme com o comando `java -version` no terminal. Se não tiver, peça ajuda ao professor antes de continuar.

2

IntelliJ IDEA instalado

Pode ser a versão Community (gratuita) ou Ultimate. É o programa (IDE) que usaremos para escrever e rodar código Java.

3

Projeto ola-mundo recebido

O arquivo `ola-mundo.zip` deve estar salvo em uma pasta de fácil acesso, por exemplo Documentos ou a Área de Trabalho.

DICA

Se algo da lista estiver faltando, avise o professor agora — é mais rápido resolver antes de seguir para os próximos passos.

1 Extraindo o projeto ola-mundo

- O projeto foi entregue como um arquivo compactado: ola-mundo.zip
- Clique com o botão direito sobre o arquivo .zip
- Windows: escolha "Extrair tudo..." (Extract All)
- Mac: dê duplo clique para descompactar automaticamente
- Escolha uma pasta de fácil acesso, por exemplo Documentos
- Ao final, você terá uma pasta chamada ola-mundo com os arquivos do projeto dentro

```
estrutura de pastas esperada

Documentos/
├─ ola-mundo/
│   ├── pom.xml
│   ├── .gitignore
│   └── src/
│       └── main/java/org/example/
│           └── Main.java
```

DICA

Guarde o caminho completo da pasta ola-mundo — vamos precisar dele no próximo passo, dentro do IntelliJ.

2 Abrindo o IntelliJ IDEA

- Abra o IntelliJ IDEA normalmente, como qualquer outro programa
- Na primeira tela (tela de boas-vindas), você verá algumas opções:
 - New Project — cria um projeto do zero
 - Open — abre um projeto que já existe em uma pasta
 - Get from VCS — clona um projeto de um repositório Git
- Como o ola-mundo já existe na sua pasta, vamos usar a opção Open

DICA

Se o IntelliJ já estiver aberto com outro projeto, use o menu File → Open no lugar da tela de boas-vindas.

3 Selecionando a pasta do projeto

- Na janela que abrir, navegue até a pasta ola-mundo que você extraiu no Passo 1
- Selecione a pasta ola-mundo (a pasta inteira, não um arquivo dentro dela) e clique em OK
- O IntelliJ vai reconhecer automaticamente que é um projeto Maven, por causa do arquivo pom.xml
- Pode aparecer uma janela perguntando se você confia neste projeto (Trust Project) — clique em Trust Project

DICA

O que é o pom.xml? É o arquivo de configuração do Maven, a ferramenta que organiza as dependências e a forma como o projeto Java é construído. Vamos falar mais dele em outra aula — por enquanto, só saiba que é ele quem diz ao IntelliJ "isto aqui é um projeto Java".

4 Aguardando a preparação do projeto

- Assim que o projeto abrir, o IntelliJ começa a indexar os arquivos e preparar o Maven
- Você vai ver barras de progresso na parte inferior da tela, com textos como:
 - "Indexing"
 - "Resolving dependencies"
 - "Downloading..."
- Isso pode levar de alguns segundos a poucos minutos, dependendo da internet
- Espere terminar antes de seguir — mexer no projeto antes disso pode causar erros temporários

5 Conhecendo a estrutura do projeto

- **pom.xml**
 - Arquivo de configuração do Maven: nome do projeto, versão do Java, etc.
- **src/main/java**
 - Pasta onde fica todo o código-fonte Java do projeto
- **org/example/Main.java**
 - A classe principal do nosso programa — é nela que vamos trabalhar hoje
- **target/**
 - Pasta gerada automaticamente com os arquivos compilados (.class). Não editamos nada aqui.



6 Executando o programa pela 1ª vez

- Abra o arquivo Main.java (dois cliques nele, na árvore de arquivos à esquerda)
- Procure o método main — é o ponto de partida de todo programa Java
- Ao lado da linha `public static void main`, existe um triângulo verde (▶)
- Clique nesse triângulo verde e escolha Run 'Main.main()'
- Alternativa pelo teclado: Shift + F10 (Windows/Linux) ou Control + R (Mac)
- Uma aba chamada Run vai abrir na parte inferior da tela — é o console do programa

ATENÇÃO

Se aparecer um erro de "JDK não configurado", veja o slide de solução de problemas mais adiante — é a falha mais comum nesse primeiro run.

7 Interagindo com o console

- O programa vai pedir para você digitar seu nome
- Clique dentro da área do console para garantir o foco, digite o nome e aperte Enter
- Em seguida, ele pede o sobrenome — digite e aperte Enter novamente
- O programa então imprime uma saudação e alguns cálculos automáticos

```
exemplo de execução

Digite o seu nome agora:
Ada
Digite o seu sobrenome agora:
Lovelace
Olá Ada Lovelace!
O ano atual é: 2026
O tamanho do seu nome é: 10
Seu nome é longo
Média final: 8
Nota Final: 10
Olá mundo!
```

Entendendo o código — leitura e variáveis

```
Main.java (trecho)

Scanner leitor = new Scanner(System.in);
String nome = leitor.nextLine();
String sobrenome = leitor.nextLine();
int ano = 2026;

System.out.println("Olá " + nome + " " + sobrenome + "!");
```

Scanner

- Classe pronta do Java para ler o que o usuário digita no teclado

String

- Tipo de dado usado para guardar texto (nome, sobrenome)

int

- Tipo de dado usado para guardar números inteiros (o ano)

- +

- Operador usado para juntar (concatenar) textos e variáveis

Entendendo o código — decisão, vetores e repetição

```
Main.java (trecho)

if ((nome.length()+sobrenome.length()) > 5) {
    System.out.println("Seu nome é longo");
} else { ... }

Integer[] notas = new Integer[3];
notas[0] = 10; notas[1] = 8; notas[2] = 6;

for (int i = 0; i < 3; i++) {
    soma = soma + notas[i];
}
```

if / else

- Decide qual bloco de código executar, dependendo de uma condição

Array (vetor)

- notas guarda 3 valores na mesma variável, acessados por posição [0],[1],[2]

for

- Repete o bloco enquanto i for menor que 3, somando cada nota

while

- Repete enquanto uma condição continuar verdadeira (usado logo depois no código)

Mão na massa: modifique o código

A

Mude a mensagem final

Troque o texto "Olá mundo!" dentro do método `exercicio01` por uma mensagem sua.

B

Adicione uma 4ª nota

Aumente o array `notas` para 4 posições, adicione um novo valor e ajuste o `for` e a divisão da média.

C

Mude a condição do `if`

Troque o número 5 por outro valor e veja como muda a resposta "nome longo/curto".

DICA

Depois de cada alteração, rode o programa de novo (▶) para ver o resultado. Errar é normal — é assim que se aprende a programar!

Problemas comuns ao rodar o projeto

"No JDK found" / SDK não configurado

File → Project Structure → Project → escolha um JDK instalado (versão 25 ou superior).

Botão verde não aparece

Confirme se abriu o arquivo Main.java certo e se o Maven já terminou de indexar (Passo 4).

Console não aceita digitação

Clique dentro da aba Run antes de digitar, para dar foco ao console.

Erro de compilação em vermelho

Releia a linha apontada pelo IntelliJ; normalmente é ponto e vírgula ou parênteses faltando.

PARTE 2

Git na prática

Agora vamos instalar o Git, criar acesso via SSH ao GitHub e enviar o ola-mundo para o seu primeiro repositório remoto.



Relembrando: o que é controle de versão?

- Gerenciamento do código-fonte ao longo do tempo
- Permite contribuições de diferentes programadores no mesmo projeto
- Permite desfazer alterações problemáticas
- Ajuda na resolução de conflitos de código
- O Git é o sistema de controle de versão que vamos usar, criado por Linus Torvalds

working directory

git add ↓ / git commit ↓

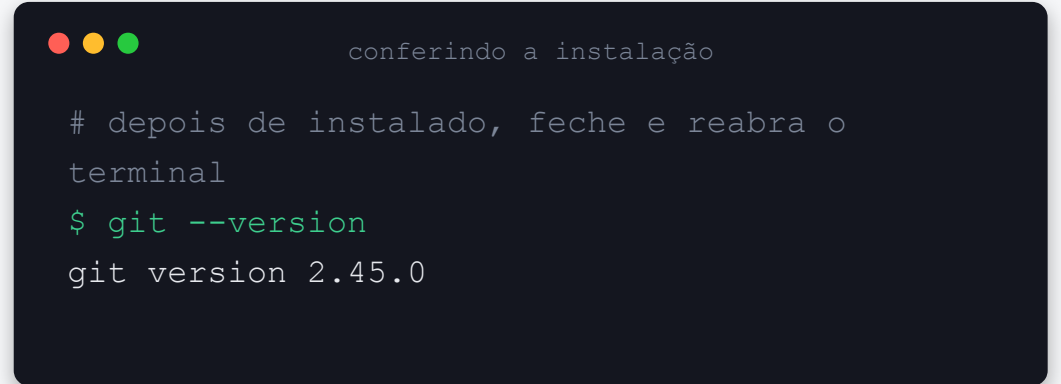
staging area

git add ↓ / git commit ↓

repository

1 Instalando o Git

- Acesse o site oficial: git-scm.com/downloads
- **Windows**
 - Baixe o instalador, clique em Next em todas as telas e finalize (as opções padrão funcionam bem)
- **Mac**
 - Abra o Terminal e rode: `git --version` — se não estiver instalado, o próprio Mac oferece para instalar as Command Line Tools
- **Linux (Ubuntu/Debian)**
 - Abra o terminal e rode: `sudo apt install git`

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar text is "conferindo a instalação". The terminal content shows a comment in Portuguese, followed by the command `git --version` and its output, `git version 2.45.0`.

```
conferindo a instalação

# depois de instalado, feche e reabra o
terminal
$ git --version
git version 2.45.0
```

DICA

Site oficial para download: git-scm.com/downloads

2 Verificando a instalação

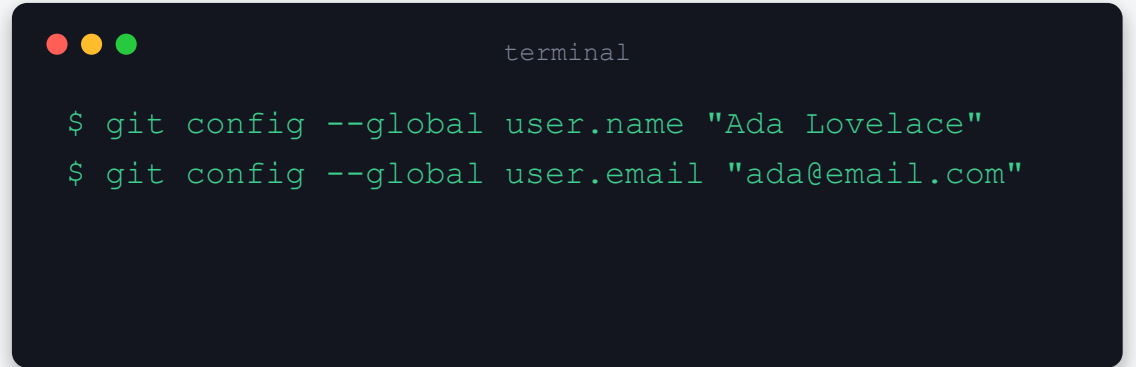
- Abra um terminal:
 - Windows: Git Bash (instalado junto com o Git) ou o Prompt de Comando
 - Mac/Linux: o Terminal padrão do sistema
- Digite o comando ao lado e aperte Enter
- Se aparecer um número de versão, o Git foi instalado com sucesso

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. The word "terminal" is written in a small, light font in the top-right corner. The terminal shows a green prompt character "\$" followed by the command "git --version" in green. Below the command, the output "git version 2.45.0" is displayed in a light gray font.

```
$ git --version
git version 2.45.0
```

3 Configurando sua identidade

- Antes do primeiro uso, o Git precisa saber quem é você — isso identifica seus commits
- Rode os dois comandos ao lado, substituindo pelo seu nome e e-mail
- O `--global` salva essa configuração para todos os projetos da sua máquina
- Isso não é login nem senha, é apenas uma identificação de quem fez a alteração

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The word "terminal" is written in a light gray font in the top-right corner. Two lines of green text represent commands entered in the terminal: "\$ git config --global user.name 'Ada Lovelace'" and "\$ git config --global user.email 'ada@email.com'".

```
terminal  
  
$ git config --global user.name "Ada Lovelace"  
$ git config --global user.email "ada@email.com"
```

4 Criando uma conta no GitHub

- Caso ainda não tenha, acesse github.com e clique em Sign up
- Informe e-mail, crie uma senha e escolha um nome de usuário
- Recomendação: escolha um nome de usuário profissional, pois ele aparecerá no seu portfólio
- Confirme seu e-mail quando o GitHub solicitar
- O GitHub é onde vamos hospedar o repositório remoto do projeto ola-mundo

DICA

Já tem conta? Ótimo — pode pular direto para o próximo passo.

5 Gerando uma chave SSH

- SSH é uma forma segura de autenticar com o GitHub sem digitar senha a cada envio
- No terminal, rode o comando ao lado (troque pelo seu e-mail do GitHub)
- Aperte Enter três vezes para aceitar o local padrão e não definir senha adicional
- Isso gera duas chaves: uma privada (fica só na sua máquina) e uma pública (vai para o GitHub)

```
terminal

$ ssh-keygen -t ed25519 -C "ada@email.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key:
[Enter]
Enter passphrase (empty for no passphrase):
[Enter]
Enter same passphrase again:
[Enter]
```

6 Cadastrando a chave no GitHub

- Copie o conteúdo da chave pública com o comando ao lado
- No GitHub, acesse: foto de perfil → Settings → SSH and GPG keys
- Clique em New SSH key
- Dê um título (ex.: "Meu notebook - aula backend") e cole a chave no campo Key
- Clique em Add SSH key para confirmar

```
terminal

# Windows (Git Bash)
$ cat ~/.ssh/id_ed25519.pub | clip

# Mac
$ pbcopy < ~/.ssh/id_ed25519.pub

# Linux
$ cat ~/.ssh/id_ed25519.pub
```

7 Testando a conexão SSH

- De volta ao terminal, rode o comando ao lado
- Na primeira vez, o Git pergunta se confia no servidor — digite yes e aperte Enter
- Se aparecer uma mensagem de "successfully authenticated" com o seu usuário, está tudo certo!

A terminal window with a dark background and light green text. The title bar shows three colored circles (red, yellow, green) and the word "terminal". The command prompt shows the command being executed, followed by the system's response.

```
terminal  
  
$ ssh -T git@github.com  
Hi ada-lovelace! You've successfully  
authenticated, but GitHub does not  
provide shell access.
```

ATENÇÃO

Não conseguiu autenticar? Confira se colou a chave pública correta (arquivo .pub) no GitHub, sem espaços extras.

8 Criando um repositório no GitHub

- No GitHub, clique no botão + no canto superior direito → New repository
- Repository name: ola-mundo
- Deixe como Public ou Private, como preferir
- NÃO marque a opção de criar README, .gitignore ou licença — nosso projeto já existe localmente
- Clique em Create repository
- Na próxima tela, copie a URL SSH (algo como `git@github.com:seu-usuario/ola-mundo.git`)

ATENÇÃO

Marcar aquelas opções criaria arquivos no GitHub que ainda não existem localmente, gerando conflito no primeiro envio.

9 Iniciando o repositório local

- No terminal, navegue até a pasta do projeto ola-mundo
- Use o comando `cd` (change directory) seguido do caminho da pasta
- Rode o comando `git init` dentro da pasta do projeto
- Isso cria uma pasta oculta `.git`, onde o Git guarda todo o histórico do projeto

```
terminal
$ cd Documentos/ola-mundo
$ git init
Initialized empty Git repository in
.../ola-mundo/.git/
```

10 Conferindo o .gitignore do projeto

- O projeto ola-mundo já vem com um arquivo .gitignore pronto
- Ele diz ao Git quais arquivos e pastas NUNCA devem ser versionados
- **target/**
 - pasta com os arquivos compilados (gerados automaticamente pelo Maven)
- **.idea/ (parcialmente)**
 - configurações pessoais do IntelliJ, que não precisam ser compartilhadas

```
.gitignore (trecho)

target/
!.mvn/wrapper/maven-wrapper.jar

### IntelliJ IDEA ###
.idea/modules.xml
.idea/jarRepositories.xml
*.iws
*.iml
```

11 Adicionando e commitando os arquivos

- **git add .**
 - Envia todos os arquivos modificados (exceto os ignorados) para a staging area
- **git commit -m "mensagem"**
 - Salva uma versão do projeto no repositório local, com uma mensagem descritiva
- Use mensagens curtas e claras, como "Primeiro commit do projeto ola-mundo"

```
terminal

$ git add .
$ git commit -m "Primeiro commit do projeto ola-mundo"
[main (root-commit) 4f2a1b0] Primeiro commit...
5 files changed, 78 insertions(+)
```

12 Vinculando o remoto e enviando

- **git remote add origin <url>**
 - Cria um atalho chamado origin apontando para o repositório do GitHub
- **git push -u origin main**
 - Envia os commits locais para o GitHub. O -u lembra essa ligação para os próximos push

```
terminal
$ git remote add origin \
  git@github.com:usuario/ola-mundo.git
$ git push -u origin main
Enumerating objects: 6, done.
...
branch 'main' set up to track 'origin/main'.
```

DICA

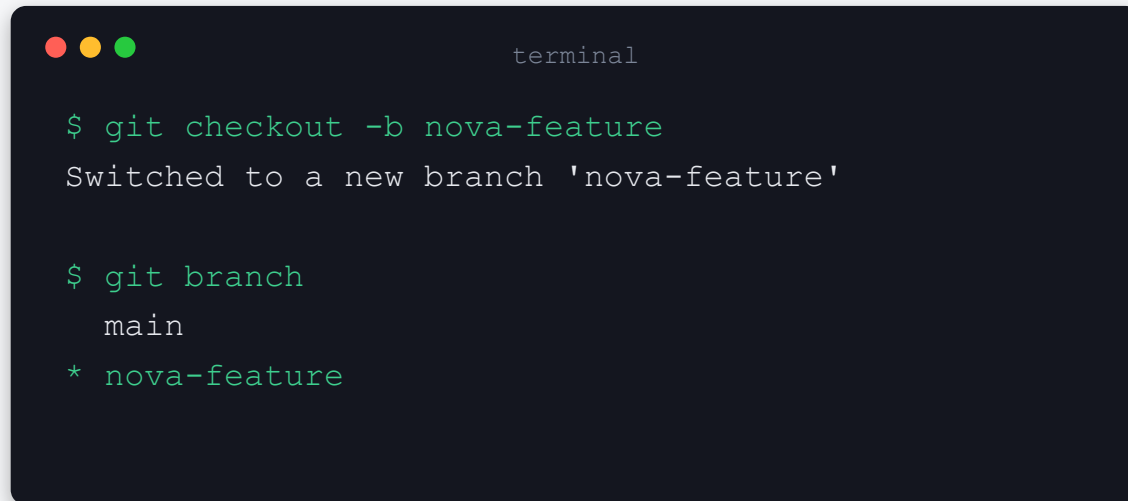
Se o branch padrão do seu projeto se chamar master em vez de main, troque no comando de acordo (git push -u origin master).

Conferindo o resultado no GitHub

- Volte ao navegador e atualize a página do seu repositório ola-mundo
- Você deve ver os arquivos do projeto: pom.xml, .gitignore e a pasta src
- Clique no commit para conferir a mensagem que você escreveu
- Parabéns — esse é o seu primeiro projeto versionado e publicado no GitHub!

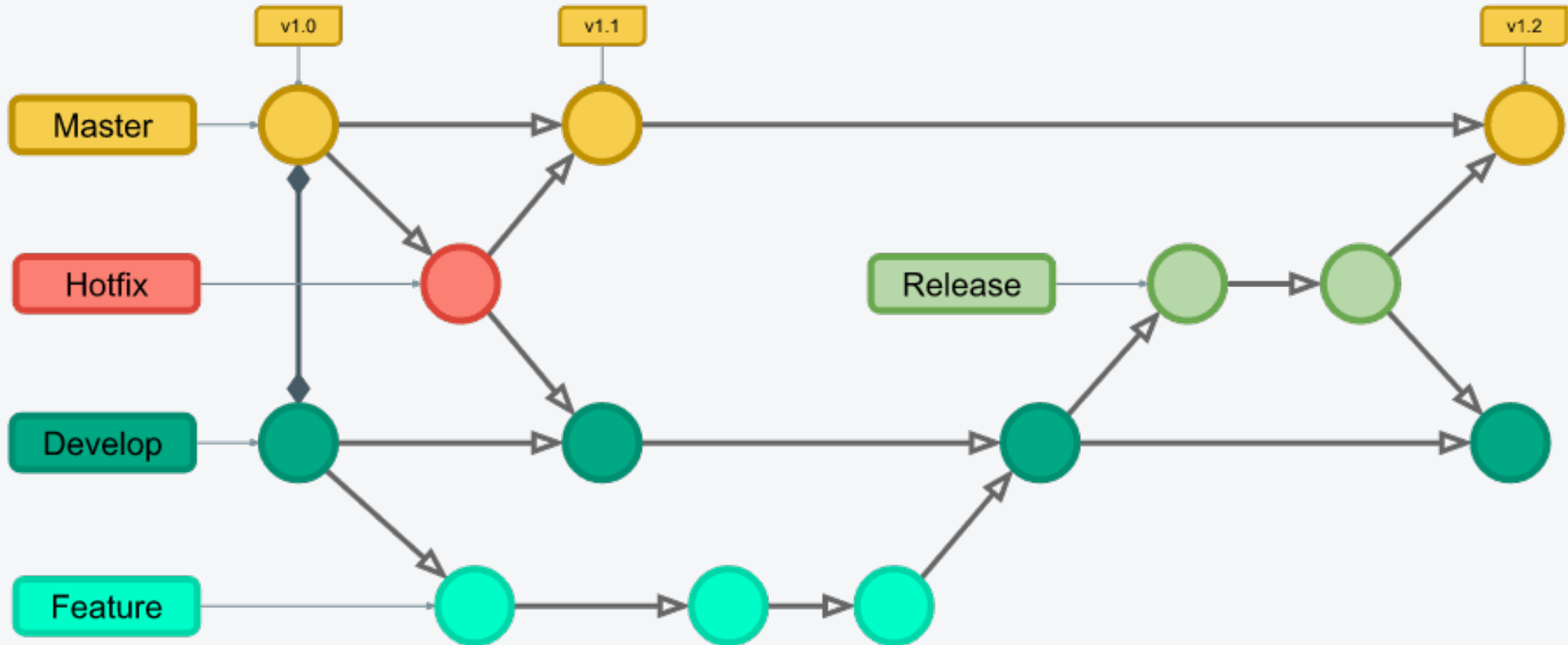
Branches na prática

- Um branch é uma linha paralela de desenvolvimento, isolada da principal (main)
- Útil para testar uma alteração sem arriscar o código que já funciona
- Crie e já mude para um novo branch com um único comando: `git checkout -b`
- Depois de testar, volte para o main com `git checkout main`

A terminal window with a dark background and light green text. It shows the execution of two git commands. The first command creates a new branch and switches to it. The second command lists the current branches, showing 'main' and the newly created 'nova-feature' branch, which is marked with an asterisk to indicate it is the current branch.

```
terminal  
  
$ git checkout -b nova-feature  
Switched to a new branch 'nova-feature'  
  
$ git branch  
main  
* nova-feature
```

Branches na prática



Unindo alterações com merge

- O comando merge incorpora as modificações de um branch em outro
- Primeiro, volte para o branch que vai receber as alterações (geralmente main)
- Depois, rode git merge apontando para o branch de origem

A terminal window with a dark background and light green text. It shows the execution of two git commands: 'git checkout main' and 'git merge nova-feature'. The output shows a fast-forward merge from 4f2a1b0 to 9c3d2e1.

```
terminal  
  
$ git checkout main  
$ git merge nova-feature  
Updating 4f2a1b0..9c3d2e1  
Fast-forward
```

Atualizando e enviando alterações

- **git fetch**
 - Atualiza as referências locais do repositório remoto, sem alterar seus arquivos
- **git pull**
 - Atualiza as referências e já aplica (merge) as mudanças nos seus arquivos locais
- **git push**
 - Envia seus commits locais para o repositório remoto no GitHub

O fluxo completo, do zero ao GitHub



O QUE APRENDEMOS HOJE

- Abrir e executar um projeto Java real no IntelliJ IDEA
- Ler e entender um código com variáveis, decisão, vetores e repetição
- Instalar o Git e configurar acesso seguro via SSH ao GitHub
- Fazer o primeiro commit e enviar um projeto para um repositório remoto
- Trabalhar com branches, merge e sincronização (fetch/pull/push)

Próxima aula: Orientação a Objetos I